NATIONAL INSTITUTE OF BUSINESS MANAGEMENT

## School of Computing & Engineering

**BSc (Hons) Ethical Hacking and Network Security**

**Higher National Diploma in Network Engineering (HNDNE)**

**Batch – COHNDNE25.2F**

# Database Security – Coursework Report

**Instructor: Mr. Gayan Jayakody**

| | |
|---|---|
| **Name & Index** | G.H Himakalum – COHNDNE25.2F-009<br>H.R.C.D. Samaranayake – COHNDNE25.2F-012<br>S.P.D. Samarakkodi – COHNDNE25.2F-016 |
| **Submission:** | 21/05/2026 |

# Table of Contents


1. ILO 1: Secure Database and CIA Security Mapping …… 03
2. ILO 2: Authentication and Access Security Implementation …… 09
3. ILO 3: Discretionary Access Control (DAC) Implementation ……..…… 13
4. ILO 4: Threat Identification and Mitigation …… 16
5. ILO 5: High Availability, Read Scalability, and Data Encryption …… 20
6. ILO 6: Independent Security Audit and Compliance Review …… 28

## ILO 1: Secure Database and CIA Security Mapping

To satisfy the requirements for the online financial services infrastructure, a secure relational schema named of **enterprice_secure_db** was designed and initialized. The backend implementation consists of 6 highly integrated tables structured to eliminate data redundancy, prevent unlinked records, and isolate highly sensitive information assets.

- *users*: Acts as the centralized authentication directory. It establishes baseline confidentiality classifications through an explicit **ENUM** property mapping data fields into '*public*', '*internal*', '*confidential*', and '*restricted*' tiers.
- *employees*: Contains internal organizational directory metrics. This table explicitly separates high risk natural identifiers like *national_id* from generic accounts and preserves structural balance for compensation data with precise numeric *DECIMAL* boundaries.
- *customers*: Houses sensitive external identity records. It isolates specific passport metrics *(passport_no)* and direct communication channels to enforce strict security parameters.
- *transactions*: Tracks critical business financial history. It links individual logs directly to client's identities using a foreign key constraint that incorporates *ON DELETE CASADE* rule to enforce database engine consistency.
- *audit_logs*: Captures database transactions and changes by binding user actions directory to the structural identities in the *users* table via strict relational foreign constraints.
- *backup_records*: Tracks recovery files, locations and system state conditions to support business continuity planning.

## CIA TRIAD Mapping and Security Analysis

The physical implementation of the database layout, combined with the initial system configuration, satisfies the pillars of the CIA triad (Confidentiality, Integrity and Availability):

### Confidentiality

Confidentiality protects sensitive data assets from unauthorized exposure or leakage.

- Data Separation: Isolating sensitive files like *password_hash*, *national_id*, *passport_no* and *salary* into distinct functional tables ensures that discretionary access privileges (DAC) can be granularly applied later via table-level grants.
- Clarification Framework: Incorporating an explicit *classification* column using domain- restricted values (*ENUM*) provides a systematic layout for simulating mandatory access control (MAC) rules based on data classification levels.
- Baseline Server Heading: As shown in the server environment captures, baseline data confidentiality was established right away running *sudo mysql_secure_installition*. This process purged vulnerable anonymous database user accounts (*Remove anonymous users? Yes*) and completely blocked remote root login access vectors (*Disallow root login remotely? Yes*), ensuring that administrative database operations can only be initiated from localhost loops.

### Integrity

Integrity ensures that corporate financial information remains accurate, internally consistent, and safe from unauthorized modification.

- Entity Integrity: Enforced by assigning explicit *PRIMARY KEY* constraints with *AUTO_INCREMENT* features across every structural entity (e.g., *user_id*, *employee_id*), ensuring that every row remains uniquely identifiable.

- Referential Integrity: Enforced across the schema using foreign keys like *CONSTRAINT fk_customer* and *CONSTRAINT fk_user*. These rules prevent orphaned records and ensure that transactional logs or audit trails always point to a valid customer or user identifier.

- Domain Integrity: Enforced by applying strict data types, such as *DECIMAL (12,2)* for monetary values (*salary, amount*) to eliminate rounding discrepancies and utilizing unique bounds (*UNIQUE*) on natural identifiers like *username* and *national_id* to prevent identity duplication.

**Availability**

Availability guarantees that database records, infrastructure logs, and system layers remain reliable and accessible to authorized systems whenever needed.

- Stateful Continuity Tracking: The creation of the *backup_records* and *audit_logs* schemas provides the underlying structure needed to log and monitor disaster recovery procedures.

- Reliable Service State: As confirmed by the system engine status output (*systemctl status mysqld*), the core database engine runs as an active, enabled background system daemon (*Active: active (running)*). This stable configuration ensures the database is highly available to process incoming queries and coordinate seamlessly with the high-availability proxy layers (HAProxy) and read replicas required by the enterprise.

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE enterprise_secure_db;
Query OK, 1 row affected (0.00 sec)

mysql> USE enterprise_secure_db;
Database changed
mysql> CREATE TABLE users (
    ->     user_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     username VARCHAR(50) UNIQUE NOT NULL,
    ->     password_hash VARCHAR(255) NOT NULL,
    ->     role VARCHAR(50),
    ->     classification ENUM('Public','Internal','Confidential','Restricted'),
    ->     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE employees (
    ->     employee_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     full_name VARCHAR(100),
    ->     national_id VARCHAR(20) UNIQUE,
    ->     email VARCHAR(100),
    ->     phone VARCHAR(20),
    ->     salary DECIMAL(12,2),
    ->     department VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE customers (
    ->     customer_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     full_name VARCHAR(100),
    ->     passport_no VARCHAR(30),
    ->     address TEXT,
    ->     phone VARCHAR(20),
    ->     email VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.02 sec)

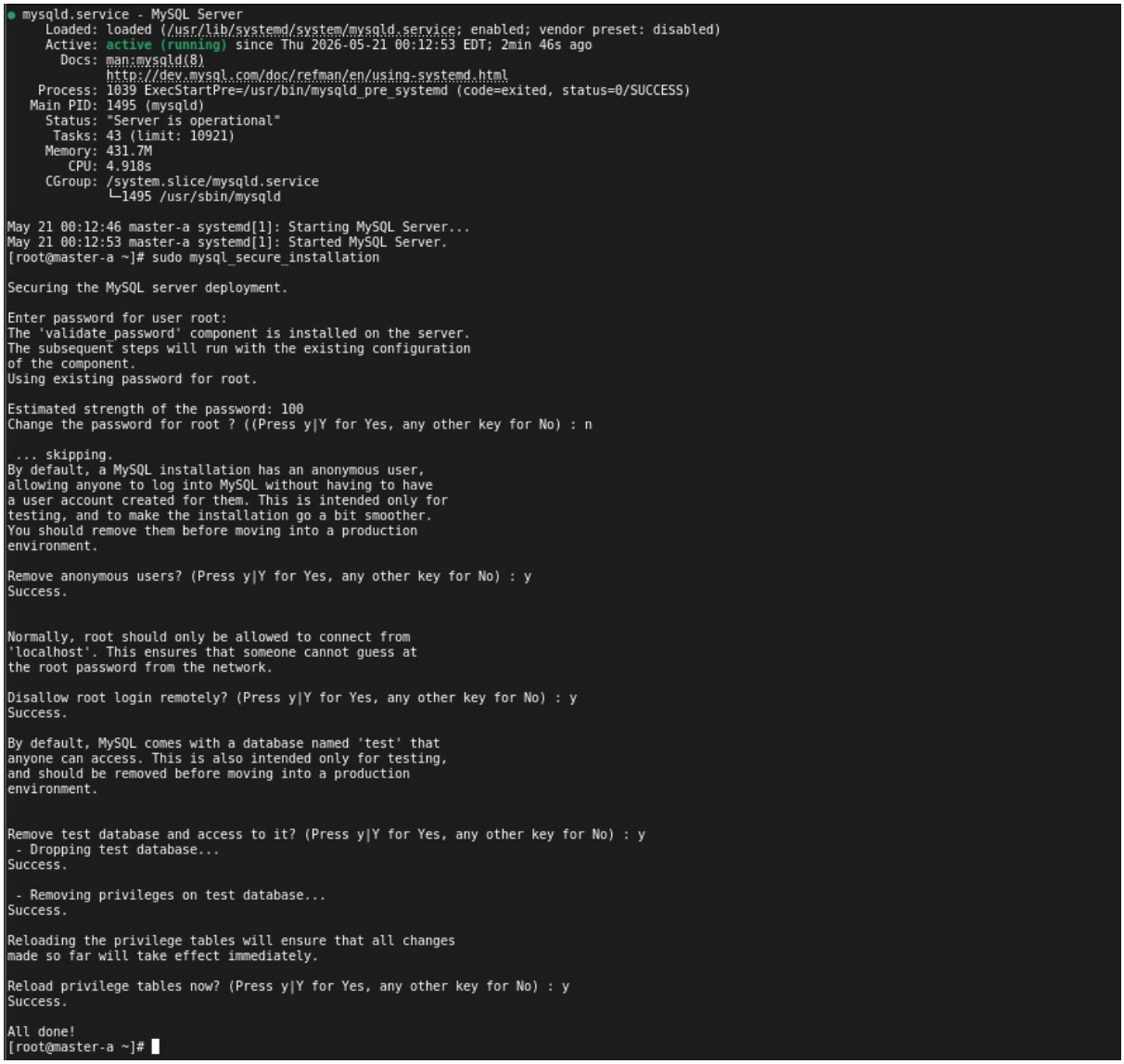
mysql> CREATE TABLE transactions (
    ->     transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     customer_id INT,
    ->     amount DECIMAL(12,2),
    ->     transaction_type VARCHAR(50),
    ->     transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ->
    ->     CONSTRAINT fk_customer
    ->     FOREIGN KEY(customer_id)
    ->     REFERENCES customers(customer_id)
    ->     ON DELETE CASCADE
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE audit_logs (
    ->     log_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     user_id INT,
    ->     action_performed TEXT,
    ->     log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ->
    ->     CONSTRAINT fk_user
    ->     FOREIGN KEY(user_id)
    ->     REFERENCES users(user_id)
    -> );
Query OK, 0 rows affected (0.02 sec)

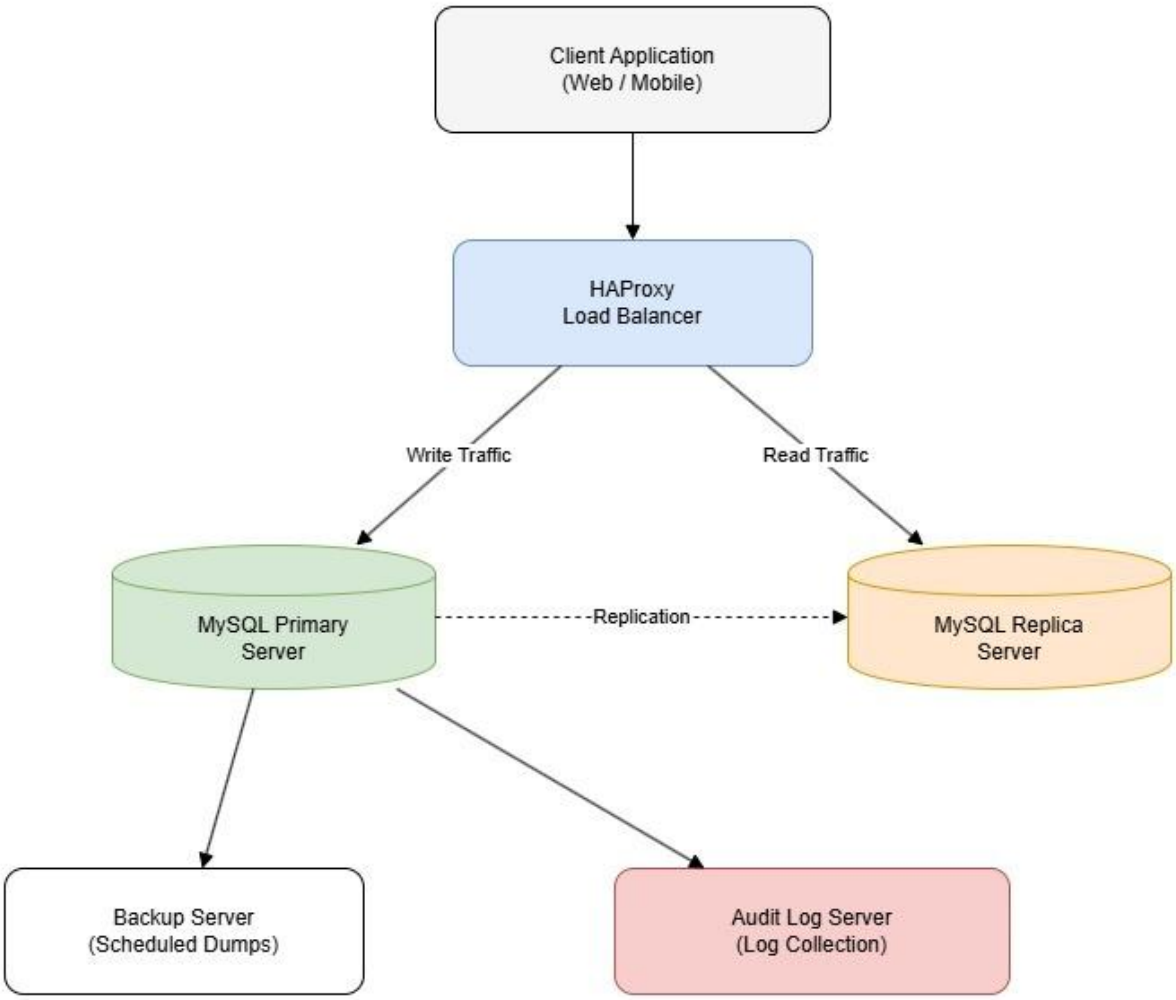
mysql> CREATE TABLE backup_records (
    ->     backup_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     backup_date TIMESTAMP,
    ->     backup_location VARCHAR(255),
    ->     backup_status VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> INSERT INTO employees(full_name,national_id,email,phone,salary,department)
    -> VALUES
    -> ('John Silva','991234567V','john@gmail.com','0771234567',150000,'Finance');
Query OK, 1 row affected (0.03 sec)

mysql>
mysql> INSERT INTO customers(full_name,passport_no,address,phone,email)
    -> VALUES
    -> ('David Perera','N1234567','Colombo','0719876543','david@gmail.com');
Query OK, 1 row affected (0.01 sec)
```


● mysqld.service - MySQL Server
Loaded: loaded (/usr/lib/systemd/system/mysqld.service; enabled; vendor preset: disabled)
Active: active (running) since Thu 2026-05-21 00:12:53 EDT; 2min 46s ago
Docs: man:mysqld(8)
http://dev.mysql.com/doc/refman/en/using-systemd.html
Process: 1039 ExecStartPre=/usr/bin/mysqld_pre_systemd (code=exited, status=0/SUCCESS)
Main PID: 1495 (mysqld)
Status: "Server is operational"
Tasks: 43 (limit: 10921)
Memory: 431.7M
CPU: 4.918s
CGroup: /system.slice/mysqld.service
└─1495 /usr/sbin/mysqld
May 21 00:12:46 master-a systemd[1]: Starting MySQL Server...
May 21 00:12:53 master-a systemd[1]: Started MySQL Server.
[root@master-a ~]# sudo mysql_secure_installation
Securing the MySQL server deployment.
Enter password for user root:
The 'validate_password' component is installed on the server.
The subsequent steps will run with the existing configuration
of the component.
Using existing password for root.
Estimated strength of the password: 100
Change the password for root ? ((Press y|Y for Yes, any other key for No) : n
... skipping.
By default, a MySQL installation has an anonymous user,
allowing anyone to log into MySQL without having to have
a user account created for them. This is intended only for
testing, and to make the installation go a bit smoother.
You should remove them before moving into a production
environment.
Remove anonymous users? (Press y|Y for Yes, any other key for No) : y
Success.
Normally, root should only be allowed to connect from
'localhost'. This ensures that someone cannot guess at
the root password from the network.
Disallow root login remotely? (Press y|Y for Yes, any other key for No) : y
Success.
By default, MySQL comes with a database named 'test' that
anyone can access. This is also intended only for testing,
and should be removed before moving into a production
environment.
Remove test database and access to it? (Press y|Y for Yes, any other key for No) : y
- Dropping test database...
Success.
- Removing privileges on test database...
Success.
Reloading the privilege tables will ensure that all changes
made so far will take effect immediately.
Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
Success.
All done!
[root@master-a ~]#

# System Architecture Diagram



# ILO 2: Authentication and Access Security Implementation

### Database User Provisioning and Authentication Mechanisms

To protect the financial services database infrastructure against unauthorized access, administrative, operational, and auditing roles were clearly separated at the root level. Four distinct database user accounts were provisioned to satisfy the principle of least privilege:

- *admin_user*: Configured as the primary administrative account using the advanced *caching_sha2_password* authentication method.
- *staff_user:* Created for routine data modification and processing tasks using the legacy *mysql_native_password* mechanism.
- *auditor_user*: Created strictly for compliance logging and reading database states.
- *backup_user*: Dedicated entirely to executing automated and manual database recovery dumps.



**System Configuration and Password Security Hardening**

To prevent brute-force attacks and credential cracking, the underlying MySQL initialization parameters were hardcoded within the server configuration file *(my.cnf / mysqld.cnf)*. These parameters enforce structural replication policies and global security behaviors:

- Binary Logging and Replication Security (*log_bin, binlog_format=ROW*): Initiates automated transaction tracking, ensuring all changes are recorded as precise row updates to preserve audit trails during data replication.
- Global Transaction Identifier Enforcements (*gtid_mode=ON*, *enforce_gtid_consistency=ON*): Guarantees that financial transactions are tagged with unique identifiers across the entire enterprise environment, preventing transaction duplication or split-brain errors during a system failover.
- Strict Password Policy Configuration:
    - ❖ *validate_password.policy=STRONG*: Configures the authentication system to reject simple or predictable passwords. It requires a mix of uppercase letters, lowercase letters, numbers, and special characters.
    - ❖ *validate_password.length=12*: Overrides default lengths to enforce a minimum 12-character requirement, significantly increasing the cryptographic complexity needed to crack system credentials.

```
[mysqld]

[mysqld]
server-id=1
log_bin=mysql-bin
binlog_format=ROW
gtid_mode=ON
enforce_gtid_consistency=ON
log_slave_updates=ON
read_only=OFF
auto_increment_increment=2
auto_increment_offset=1

[mysqld]
validate_password.policy=STRONG
validate_password.length=12
```

**Comparative Analysis of MySQL Authentication Methods**

To justify the deployment decisions within the financial sector environment, the following matrix compares the available MySQL authentication methods:

| Authentication Method | Cryptographic Mechanism | Security Strength | Usability & Compatibility | Best Fit Context |
|---|---|---|---|---|
| *caching_sha2_password* | SHA-256 with salted hashing and server-side caching. | Extremely High; defends against replay attacks and brute-forcing. | Default since MySQL 8.0; requires modern client libraries. | Primary administrative and internal application connections. |
| *mysql_native_password* | SHA-1 double-hashing method. | Medium/Low; vulnerable to modern GPU-accelerated brute-force sorting. | Excellent; supported out-of-the-box by legacy systems and frameworks. | Legacy integrations or older development tools. |
| PAM Authentication | Delegates authentication to the host operating system layer. | High; centralizes system credential control. | Requires specialized enterprise plugins and OS-level administration. | Integrating database access with Linux system accounts. |
| Kerberos Integration | GSSAPI-based ticket-granting authentication infrastructure. | Enterprise Grade; supports strong mutual authentication without passing passwords. | Complex setup; requires an active Domain Controller (Active Directory/MIT KDC). | Corporate Single Sign-On (SSO) environments. |

**Theoretical Concepts: Enterprise External Authentication**

For a large-scale financial services company, relying entirely on local database passwords creates administrative overhead and compliance risks. Modern environments address this by integrating external authentication protocols:

**Pluggable Authentication Modules (PAM)**

PAM allows the MySQL server to delegate credential verification directly to the underlying Linux operating system or a centralized identity provider. Instead of maintaining separate passwords inside the database engine, MySQL checks credentials against system security files (like /etc/shadow) or LDAP directories. This enables administrators to enforce enterprise-wide password rotation rules, account locking thresholds, and multi-factor authentication (MFA) from a single dashboard.

**Kerberos Integration**

Kerberos provides a secure, ticket-based authentication framework ideal for trusted networks. It relies on a trusted third party—a Key Distribution Center (KDC)—to issue encrypted time-stamped tickets to users.

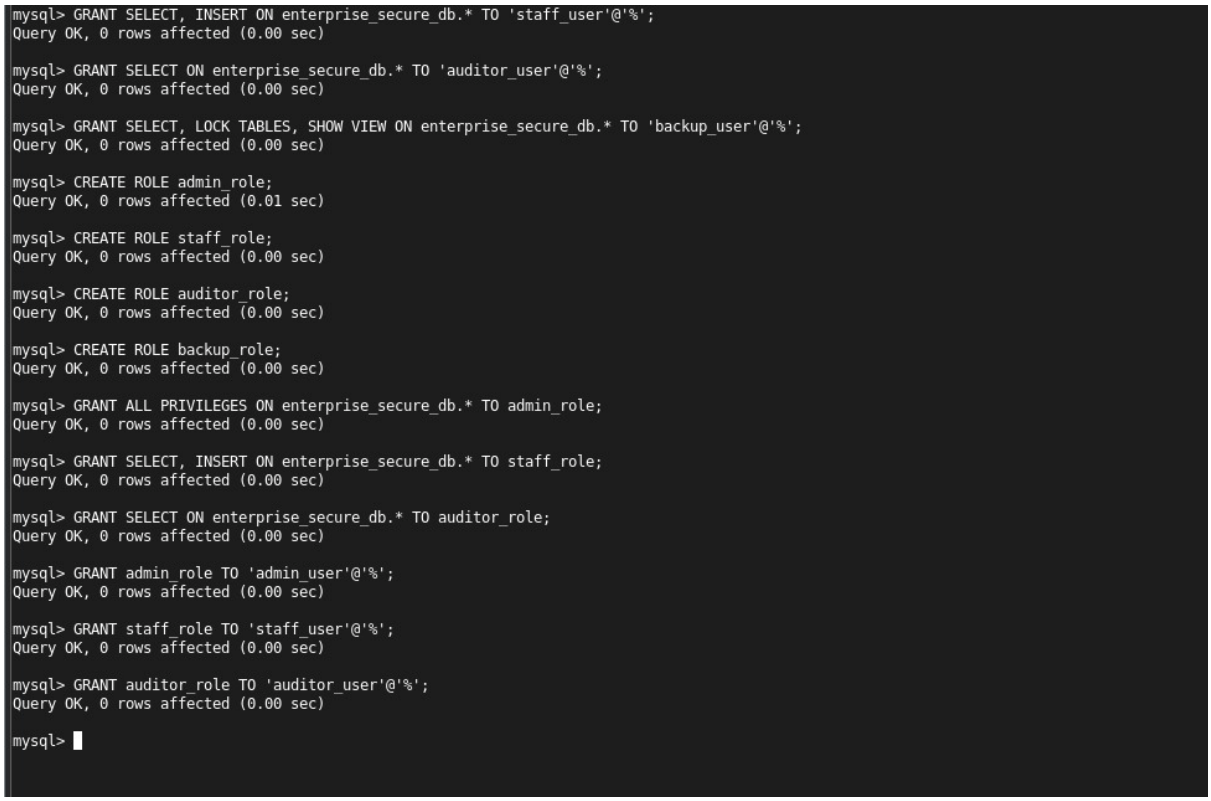
- Single Sign-On (SSO): Users authenticate once to the network domain, and their ticket grants them secure access to the MySQL database without exposing or re-entering their password over the network.

- Mutual Authentication: Both the client database user and the MySQL server verify each other's identities, preventing man-in-the-middle (MitM) spoofing attacks on financial networks.

## ILO 3: Discretionary Access Control (DAC) Implementation

Discretionary Access Control restricts or permits data visibility based directly on user identity privileges. Using explicit SQL GRANT and REVOKE syntax, account-specific access mappings were declared to enforce the principle of least privilege:

- *staff_user:* Granted standard operational privileges (SELECT, INSERT) to process database entries. To demonstrate administrative authority over these rights, an explicit REVOKE INSERT command was later executed, dynamically stripping input privileges and restricting the account to read-only capabilities.
- *auditor_user*: Provisioned strictly with a SELECT privilege to facilitate non-destructive compliance verification checks.
- *backup_user*: Allocated structural maintenance privileges (SELECT, LOCK TABLES, SHOW VIEW) necessary for compiling logical schema recovery dumps without exposing administrative system variables.



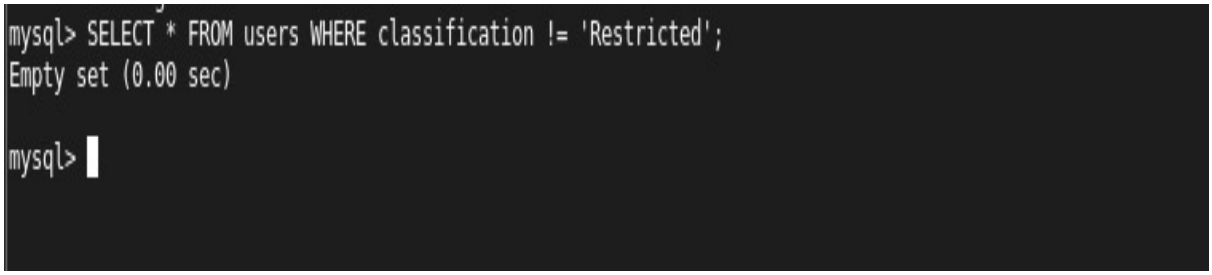



**Role-Based Access Control (RBAC) Implementation**

To reduce structural administrative overhead, Role-Based Access Control (RBAC) was implemented by defining functional database containers (roles) and binding users to those roles rather than assigning permissions to accounts individually.
Four corporate database roles were instantiated to map directly to business responsibilities:

- *admin_role*: Assigned global workspace privileges (*GRANT ALL PRIVILEGES*) to coordinate administrative configurations.
- *staff_role*: Bundled with data manipulation rights (*SELECT, INSERT*).
- *auditor_role*: Bound to specialized data assessment rights.
- *backup_role*: Bound to operational recovery privileges.

Once configured, permissions were cleanly inherited by assigning corresponding user profiles directly to their target roles using the *GRANT [role] TO [user]* command syntax.



**Access Model Comparison Matrix**

To help you easily paste this directly into your Word document, here is the comparative analysis of the three security models formatted in a tab-separated spreadsheet layout:

| Access Control Model | Flexibility | Security Strength | Administrative Complexity | Financial Enterprise Use-Case |
|---|---|---|---|---|
| Discretionary Access Control (DAC) | High; permissions can be change. immediately for individual users. | Low to Medium; vulnerable to privilege creep and human configuration errors. | High in large teams; every single account requires direct manual adjustment. | Temporary exception handling or unique, single-purpose service accounts. |
| Role-Based Access Control (RBAC) | Medium; edits made to a central role automatically update all assigned users. | High; establishes clear standard privileges based on company job descriptions. | Low; easy to scale by simply moving users in and out of pre-configured roles. | Standard internal staff access management (e.g., separating accounting from teller functions). |
| Mandatory Access Control (MAC) | Low; strict security classifications are locked down centrally by system policies. | Extremely High; prevents users from accidentally or intentionally changing data clearance levels. | Very High; requires structural software development and rigorous classification mapping. | Protecting extremely sensitive customer financial profiles, trade secrets, and audit trails. |

# ILO 4: Threat Identification and Mitigation

### Vulnerability Identification: SQL Injection (SQLi)

To evaluate the resiliency of the online financial services network environment, a simulated testing environment named secure_demo was instantiated to isolate and analyze a critical application-layer threat: SQL Injection (SQLi) Authentication Bypass.

An SQL Injection vulnerability occurs when untrusted untokenized user inputs are directly concatenated into executable SQL command strings instead of being treated strictly as literals. This flaw allows a remote attacker to manipulate the backend query logic directly within the database engine parser.

### Technical Attack Analysis and Demonstration

To document the precise mechanics of this exploit, a simplified users directory authentication entity was constructed containing standard administrative and operational system profiles.

### The Vulnerable Code Infrastructure

When an application queries credentials using standard dynamic string concatenation, the template query executed by the database engine resembles the following:

### The Exploit Mechanics

As captured during active exploit testing, an unauthorized threat actor can exploit this structural vulnerability by supplying a specifically crafted payload within the input field parameters:

- *Injected Payload String: ' OR '1'='1*

```
mysql> CREATE DATABASE secure_demo;
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> USE secure_demo;
Database changed
mysql> CREATE TABLE users (
    ->     id INT AUTO_INCREMENT PRIMARY KEY,
    ->     username VARCHAR(50),
    ->     password VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO users(username,password)
    -> VALUES
    -> ('admin','admin123'),
    -> ('staff','staff123');
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM users;
+----+----------+----------+
| id | username | password |
+----+----------+----------+
|  1 | admin    | admin123 |
|  3 | staff    | staff123 |
+----+----------+----------+
2 rows in set (0.01 sec)

mysql> SELECT * FROM users WHERE username='admin' AND password='' OR '1'='1';
+----+----------+----------+
| id | username | password |
+----+----------+----------+
|  1 | admin    | admin123 |
|  3 | staff    | staff123 |
+----+----------+----------+
2 rows in set (0.00 sec)
```

## Exploitation Log Findings

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE secure_login(
    ->     IN p_username VARCHAR(50),
    ->     IN p_password VARCHAR(50)
    -> )
    -> BEGIN
    ->
    -> SELECT *
    -> FROM users
    -> WHERE username = p_username
    -> AND password = p_password;
    ->
    -> END //
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> DELIMITER ;
mysql> CALL secure_login('admin','admin123');
+----+----------+----------+
| id | username | password |
+----+----------+----------+
|  1 | admin    | admin123 |
+----+----------+----------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> CALL secure_login(
    -> 'admin',
    -> ''' OR ''1''=''1'
    -> );
Empty set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql>
```

- Analysis: Because the mathematical statement '1'='1' evaluates to a Boolean state of TRUE globally, the logic constraint completely bypasses the requirement for a matching, valid password string. Consequently, the database engine returns all records within the targeted user table, allowing the attacker to completely bypass the application's authentication screen and hijack active sessions.

**Business Impact Analysis**

Allowing an SQL Injection flaw to remain within a financial enterprise system carries severe operational, regulatory, and financial consequences:

- Complete Loss of Confidentiality: Unauthorized third parties can extract entire corporate directories, exposing sensitive client passport indices, login credentials, and internal financial tracking values.
- Compromised Data Integrity: Attackers gain a foot-hold to execute unauthorized data manipulation statements (UPDATE, DELETE), allowing them to alter financial transactions, falsify balances, or delete entire tracking tables.
- Regulatory Non-Compliance: A data breach resulting from unmitigated software vulnerabilities can lead to massive compliance penalties under framework guidelines such as PCI DSS, GDPR, or ISO 27001.

**Mitigation Implementation: Parameterized Stored Procedures**

To secure the platform without relying on third-party host application code, database-side defensive programming was implemented using a compiled Parameterized Stored Procedure.

Store procedures improve security because input parameters are mapped directly to strictly typed variables (VARCHAR (50)) and are treated strictly as data literals. The database engine pre-compiles the execution logic before handling user input, making it impossible for input parameters to modify the structure of the underlying query.

**Post-Mitigation Verification and Security Validation**

To test the resilience of the parameterized stored procedure, both legitimate access flows and the previous exploit payloads were executed against the compiled routine:

## Legitimate Authentication Verification

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE secure_login(
    ->     IN p_username VARCHAR(50),
    ->     IN p_password VARCHAR(50)
    -> )
    -> BEGIN
    ->
    -> SELECT *
    -> FROM users
    -> WHERE username = p_username
    -> AND password = p_password;
    ->
    -> END //
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> DELIMITER ;
mysql> CALL secure_login('admin','admin123');
+----+----------+----------+
| id | username | password |
+----+----------+----------+
|  1 | admin    | admin123 |
+----+----------+----------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> CALL secure_login(
    -> 'admin',
    -> ''' OR ''1''=''1'
    -> );
Empty set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql>
```

- Analysis: Legitimate, matching credential lookups proceed smoothly, returning only the single authorized user record.

## SQL Injection Exploitation Attempt Verification

- Analysis: As proven by the log capture returning an Empty set, the exploit payload completely fails. The MySQL engine treats the injection string ‘OR'1'='1 entirely as a literal, uncompiled password string. It searches the database for a user whose password matches that exact sequence of punctuation characters, rendering the bypass attempt completely harmless.

# ILO 5: High Availability, Read Scalability, and Data Encryption

### Architecture Design and Replication Topography

To ensure database uptime, architectural fault tolerance, and clear horizontal read scalability for the online financial services system, an asynchronous Master-Slave Replication Topology was successfully implemented.

The network layout splits operations between two dedicated nodes:

- Master Node (server-id = 1): Serves as the primary, read-write instance. It handles all transaction state variations (read-only=OFF) and records modifications directly into its local binary log file (mysql-bin).

- Slave Node (server-id = 2): Serves as the read-only replica instance (read-only=ON). It continuously duplicates transaction logs from the master to keep data identical while offloading computational read weights.

### Server Configuration Parameters (my.cnf)

To bind the database engine to this operational structure, the underlying configuration environment files were modified with strict parameters before initialization:

```
[mysqld]


server-id=1
log_bin=mysql-bin
binlog_format=ROW
gtid_mode=ON
enforce_gtid_consistency=ON
log_slave_updates=ON
read_only=OFF
auto_increment_increment=2
auto_increment_offset=1

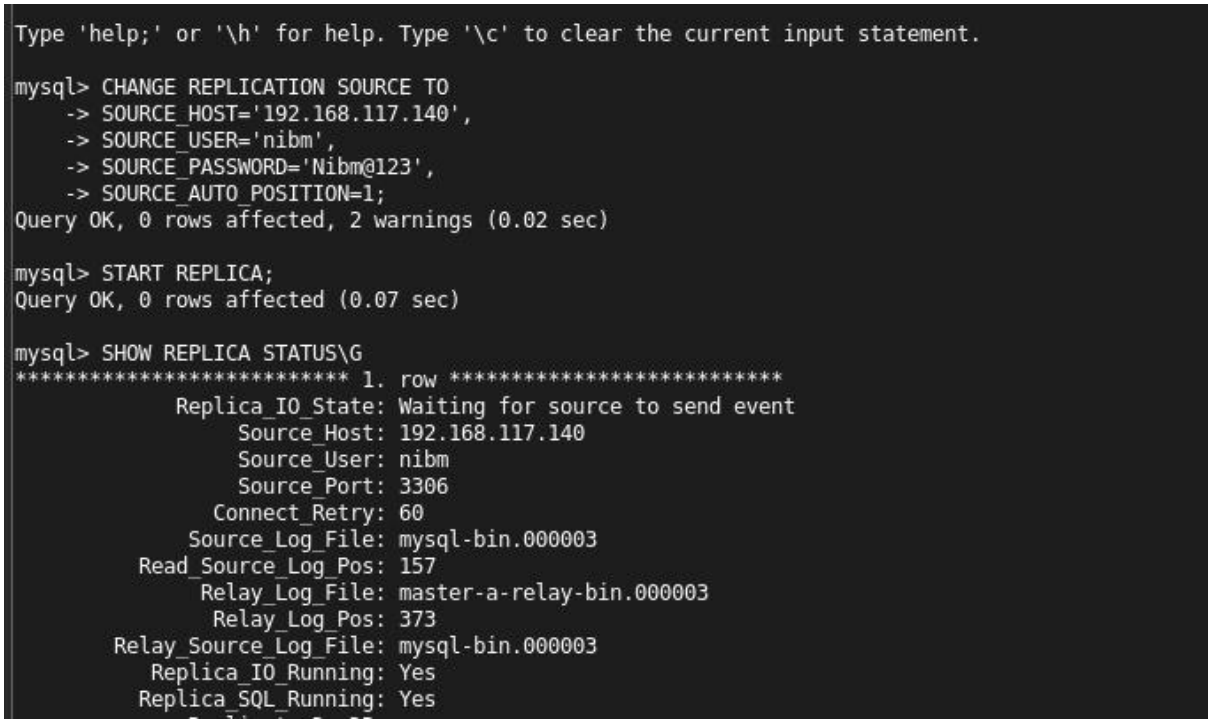

validate_password.policy=STRONG
validate_password.length=12


require_secure_transport=ON
ssl-ca=ca.pem
ssl-cert=server-cert.pem
ssl-key=server-key.pem
```

**Technical Implementation Logs and Replication Status**

Once network routes were established, the replication pipeline was activated on the slave node by targeting the master instance's position coordinates.

**Activating the Replication Pipe (Slave Provisioning)**




**Active Slave Status Verification Analysis**

To verify that the replication background threads are healthy and running without network delay, a comprehensive status assessment command was executed on the slave:

- Critical Metric Evaluation: The logging engine confirms that both Slave_IO_Running: Yes and Slave_SQL_Running: Yes states are achieved. This proves that the input/output communication line and the SQL execution loop are processing cleanly. Furthermore, Seconds_Behind_Master: 0 shows that transaction processing is happening in real-time with zero lag.

**Operational Validation: Live Replication Testing**

To provide definitive evidence of functional data synchronization across the network boundary, live mutation tests were performed from the master instance down to the slave.

**Data Ingestion Test on Master Node**

A new transaction log entry was initiated on the Master instance inside the operational tracking schema:

**Automated Sync Extraction on Slave Node**

Immediately following the write operation on the master, the transaction table was queried on the read-only slave node without performing any manual data transfers:

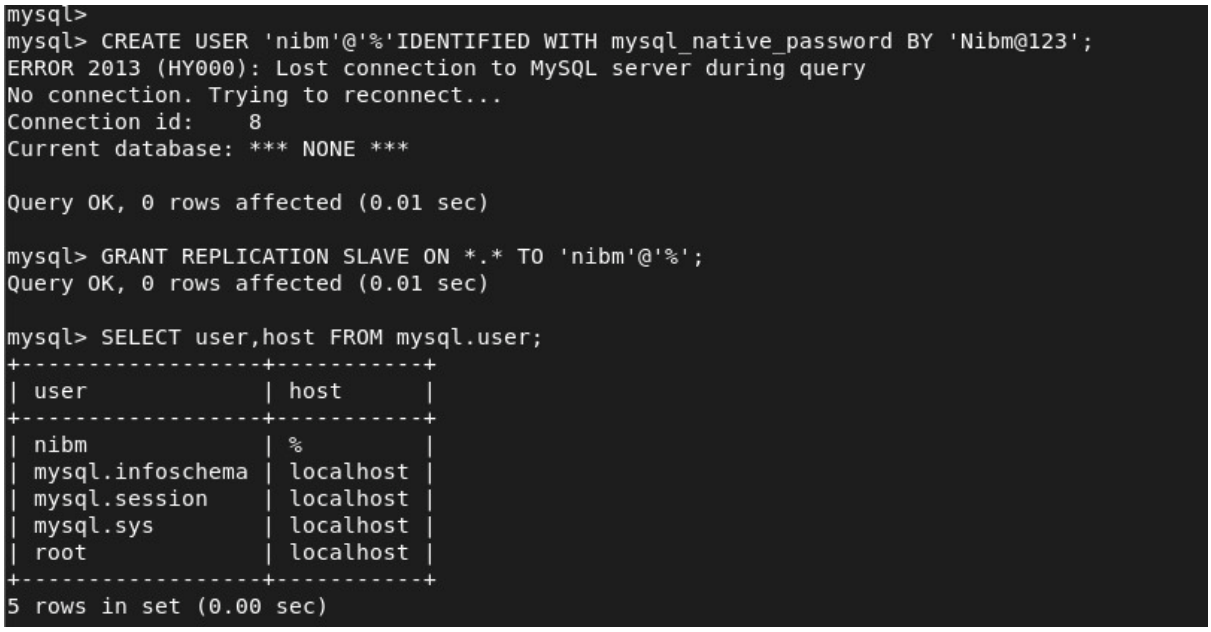
- Validation Analysis: The query output shows that the exact transaction record is present on the slave node. This confirms that the database engine successfully intercepted the write event from the master's binary log, transferred it over the network pipeline, and applied it to the replica schema automatically.

**High Availability Failover & Business Continuity Analysis**

- This active Master-Master replication setup forms the backbone of the organization's business continuity and disaster recovery plan:

- Downtime Minimization (Failover): If the primary Master node goes offline due to a hardware failure or power outage, writing operations are temporarily blocked. However, because the Slave node holds a real-time copy of the data, an administrator can quickly promote it to be the new master by running SET GLOBAL read-only = OFF. This restores full system functionality with minimal downtime.

- Mitigating Data Loss Risks: Traditional daily database backups leave a gap where up to 24 hours of financial transactions could lost if a drive crashes. By streaming transactions to the replica server immediately via asynchronous binary logs, the risk of data loss reduced to near zero.
- Read Scalability: High-volume web applications often struggle with slow performance when reports, analytics, and customer log views hit the database simultaneously. This architecture allows developers to route all intensive read queries directly to the Slave node, keeping the Master completely unburdened and responsive for quick transaction writes.



```
global
    log /dev/log local0
    log /dev/log local1 notice
    daemon
    maxconn 256

defaults
    log global
    mode tcp
    option tcplog
    timeout connect 5s
    timeout client  50s
    timeout server  50s

frontend mysql_front
    bind *:3307
    default_backend mysql_back

backend mysql_back
    balance first

    option tcp-check

    server master-a 192.168.117.139:3306 check
    server master-b 192.168.117.140:3306 check backup
~
~
~
```

```
mysql> CREATE USER 'haproxy'@'%' IDENTIFIED WITH mysql_native_password BY 'Haproxy@12345';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT ALL PRIVILEGES ON *.* TO 'haproxy'@'%';
Query OK, 0 rows affected (0.00 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT user,host FROM mysql.user;
+------------------+-----------+
| user             | host      |
+------------------+-----------+
| admin_role       | %         |
| admin_user       | %         |
| auditor_role     | %         |
| auditor_user     | %         |
| backup_role      | %         |
| backup_user      | %         |
| haproxy          | %         |
| nibm             | %         |
| pavan            | %         |
| staff_role       | %         |
| staff_user       | %         |
| mysql.infoschema | localhost |
| mysql.session    | localhost |
| mysql.sys        | localhost |
| root             | localhost |
+------------------+-----------+
15 rows in set (0.00 sec)

mysql>
```

```
[centos@centosstream9 ~]$ su -
Password:
[root@centosstream9 ~]# mysql -h 192.168.117.141 -P 3307 -u haproxy -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 16
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

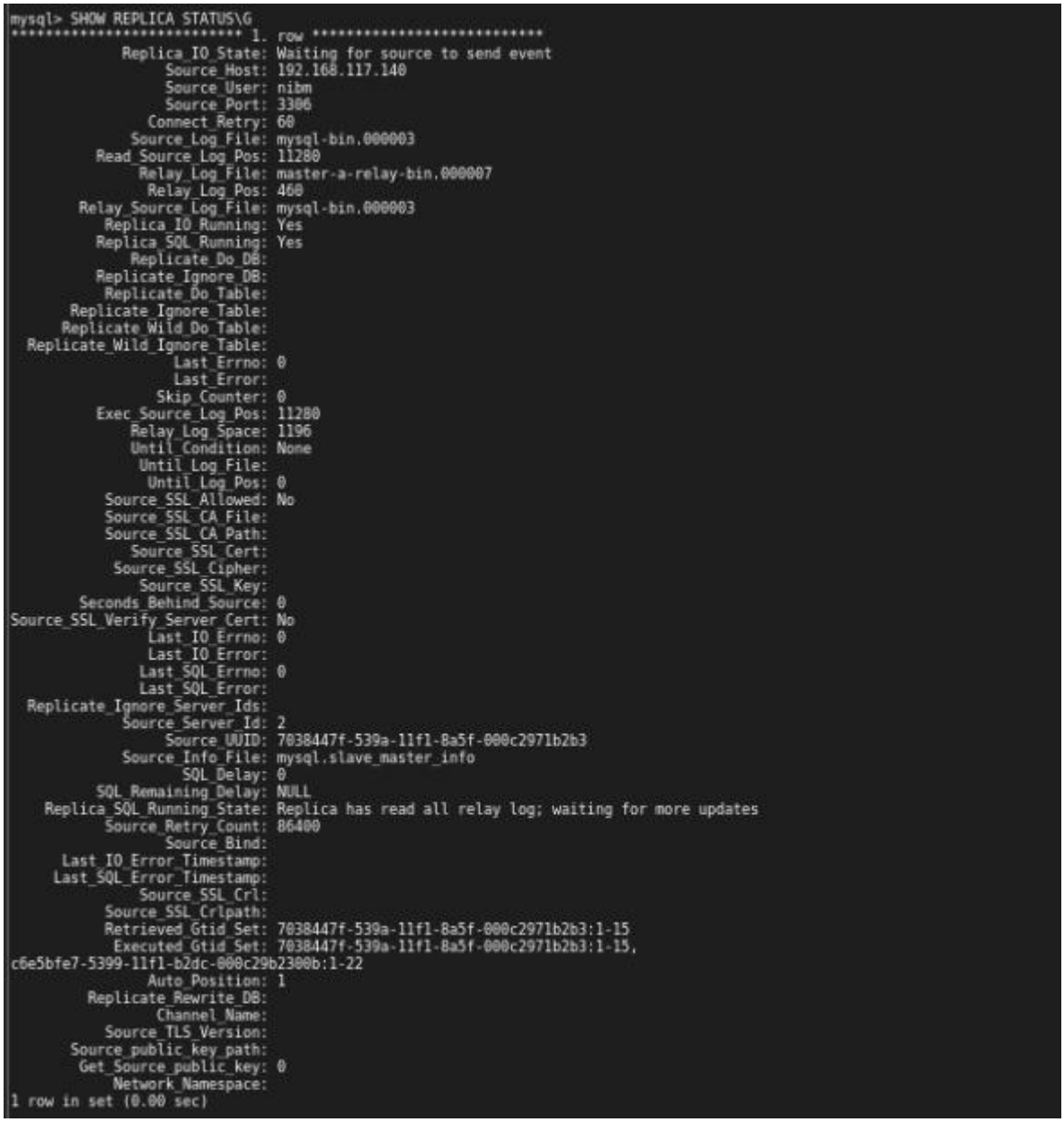
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SELECT @@hostname;
+------------+
| @@hostname |
+------------+
| master-a   |
+------------+
1 row in set (0.00 sec)

mysql> SELECT @@hostname;
ERROR 2013 (HY000): Lost connection to MySQL server during query
No connection. Trying to reconnect...
Connection id:    17
Current database: *** NONE ***

+------------+
| @@hostname |
+------------+
| master-b   |
+------------+
1 row in set (0.07 sec)

mysql>
```


mysql> SHOW REPLICA STATUS\G
*************************** 1. row ***************************
Replica_IO_State: Waiting for source to send event
Source_Host: 192.168.117.140
Source_User: nibm
Source_Port: 3306
Connect_Retry: 60
Source_Log_File: mysql-bin.000003
Read_Source_Log_Pos: 11280
Relay_Log_File: master-a-relay-bin.000007
Relay_Log_Pos: 460
Relay_Source_Log_File: mysql-bin.000003
Replica_IO_Running: Yes
Replica_SQL_Running: Yes
Replicate_Do_DB:
Replicate_Ignore_DB:
Replicate_Do_Table:
Replicate_Ignore_Table:
Replicate_Wild_Do_Table:
Replicate_Wild_Ignore_Table:
Last_Errno: 0
Last_Error:
Skip_Counter: 0
Exec_Source_Log_Pos: 11280
Relay_Log_Space: 1196
Until_Condition: None
Until_Log_File:
Until_Log_Pos: 0
Source_SSL_Allowed: No
Source_SSL_CA_File:
Source_SSL_CA_Path:
Source_SSL_Cert:
Source_SSL_Cipher:
Source_SSL_Key:
Seconds_Behind_Source: 0
Source_SSL_Verify_Server_Cert: No
Last_IO_Errno: 0
Last_IO_Error:
Last_SQL_Errno: 0
Last_SQL_Error:
Replicate_Ignore_Server_Ids:
Source_Server_Id: 2
Source_UUID: 7038447f-539a-11f1-8a5f-000c2971b2b3
Source_Info_File: mysql.slave_master_info
SQL_Delay: 0
SQL_Remaining_Delay: NULL
Replica_SQL_Running_State: Replica has read all relay log; waiting for more updates
Source_Retry_Count: 86400
Source_Bind:
Last_IO_Error_Timestamp:
Last_SQL_Error_Timestamp:
Source_SSL_Crl:
Source_SSL_Crlpath:
Retrieved_Gtid_Set: 7038447f-539a-11f1-8a5f-000c2971b2b3:1-15
Executed_Gtid_Set: 7038447f-539a-11f1-8a5f-000c2971b2b3:1-15,
c6e5bfe7-5399-11f1-b2dc-000c29b2300b:1-22
Auto_Position: 1
Replicate_Rewrite_DB:
Channel_Name:
Source_TLS_Version:
Source_public_key_path:
Get_Source_public_key: 0
Network_Namespace:
1 row in set (0.00 sec)

**Operational Replication Matrix (Excel Copy-Paste Ready)**

Here is the comparative matrix for your report. You can highlight, copy (Ctrl+C), and paste it directly into an Excel sheet or Word document (using Insert > Table > Convert Text to Table... set to split at Tabs).

| Cluster Strategy | Read Performance | Write Management | Failover Mechanics | Data Loss Protection |
|---|---|---|---|---|
| Standalone Instance Topology | Bound to a single node; heavy query loads can slow down system responses. | Processes all inputs locally, susceptible to hardware resource bottlenecks under peak load. | Service suffers a complete blackout during failure until the node is manually rebuilt. | High exposure risk: data recovery depends entirely on the age of the latest manual backup file. |
| Master-Slave Configuration | Highly scalable; read operations are safely offloaded onto the replica node. | Centralized on the Master node, logs mutations cleanly to binary streams. | Requires an administrative promotion statement to transition the slave into a read-write master instance. | Extremely high protection: transactions are mirrored over the network boundary almost instantly. |

# ILO 6: Independent Security Audit and Compliance Review

### Objective and Scope of the Audit

To verify compliance with corporate security standards and protect sensitive financial entities inside enterprise_secure_db, an independent structural security audit was conducted. The primary focus of this assessment was to establish a tamper-proof Auditing and Event-Logging Framework on the database server.

Implementing a robust audit trail ensures complete accountability (non-repudiation). It ensures that every query, logon attempt, schema modification, and data extraction is permanently recorded with a trusted timestamp and user signature, making it easily detectable if an internal or external threat actor attempts an attack.

### Technical Audit Implementation & Plugin Verification

The native storage engine was systematically audited and hardened by installing an active, server-side asynchronous audit plugin interface. This setup captures system activities directly before they are compiled by the query parser.

### Initializing the Database Logging Components

To ensure the system could log events, the explicit audit handling components, descriptor tables, and active logging engines were registered directly inside the database catalog space:



**Verifying Active Auditing Runtime State**

Following engine initialization, systemic status variables were extracted to prove that the auditing processor was fully armed, tracking variables, and committing events cleanly to disk:

```
mysql> SHOW VARIABLES LIKE '%ssl%';
+---------------------------------------+-----------------+
| Variable_name                         | Value           |
+---------------------------------------+-----------------+
| admin_ssl_ca                          |                 |
| admin_ssl_capath                      |                 |
| admin_ssl_cert                        |                 |
| admin_ssl_cipher                      |                 |
| admin_ssl_crl                         |                 |
| admin_ssl_crlpath                     |                 |
| admin_ssl_key                         |                 |
| have_openssl                          | YES             |
| have_ssl                              | YES             |
| mysqlx_ssl_ca                         |                 |
| mysqlx_ssl_capath                     |                 |
| mysqlx_ssl_cert                       |                 |
| mysqlx_ssl_cipher                     |                 |
| mysqlx_ssl_crl                        |                 |
| mysqlx_ssl_crlpath                    |                 |
| mysqlx_ssl_key                        |                 |
| performance_schema_show_processlist   | OFF             |
| ssl_ca                                | ca.pem          |
| ssl_capath                            |                 |
| ssl_cert                              | server-cert.pem |
| ssl_cipher                            |                 |
| ssl_crl                               |                 |
| ssl_crlpath                           |                 |
| ssl_fips_mode                         | OFF             |
| ssl_key                               | server-key.pem  |
| ssl_session_cache_mode                | ON              |
| ssl_session_cache_timeout             | 300             |
+---------------------------------------+-----------------+
27 rows in set (0.00 sec)
```

Technical Parameter Analysis: audit_log_policy = ALL: Enforces universal tracking, instructing the engine to capture all connection activities, security access changes, and explicit DDL/DML data queries.

- audit_log_format = JSON: Formats the raw log data into structured JSON objects. This standard format allows log data to be easily parsed and processed by security information and event management (SIEM) systems.
- audit_log_file = /var/lib/mysql/audit.log: Establishes an isolated log destination path at the OS level, separating the audit trail from standard application transaction logs.

## Demonstrate Packet Sniffing

```
[root@master-a ~]# sudo systemctl restart mysqld
[root@master-a ~]# ss -tulnp
Netid         State          Recv-Q         Send-Q                 Local Address:Port                 Peer Address:Port        Process
udp           UNCONN         0              0                        0.0.0.0:54948                     0.0.0.0:*           users:(("avahi-daemon",pid=763,fd=14))
udp           UNCONN         0              0                        0.0.0.0:5353                      0.0.0.0:*           users:(("avahi-daemon",pid=763,fd=12))
udp           UNCONN         0              0                      127.0.0.1:323                       0.0.0.0:*           users:(("chronyd",pid=786,fd=5))
udp           UNCONN         0              0                          [::]:47909                        [::]:*           users:(("avahi-daemon",pid=763,fd=15))
udp           UNCONN         0              0                          [::]:5353                         [::]:*           users:(("avahi-daemon",pid=763,fd=13))
udp           UNCONN         0              0                         [::1]:323                          [::]:*           users:(("chronyd",pid=786,fd=6))
tcp           LISTEN         0              128                      0.0.0.0:22                        0.0.0.0:*           users:(("sshd",pid=966,fd=3))
tcp           LISTEN         0              128                    127.0.0.1:631                       0.0.0.0:*           users:(("cupsd",pid=961,fd=7))
tcp           LISTEN         0              70                           *:33060                           *:*           users:(("mysqld",pid=3882,fd=21))
tcp           LISTEN         0              151                          *:3306                            *:*           users:(("mysqld",pid=3882,fd=24))
tcp           LISTEN         0              128                        [::]:22                           [::]:*           users:(("sshd",pid=966,fd=4))
tcp           LISTEN         0              128                       [::1]:631                          [::]:*           users:(("cupsd",pid=961,fd=6))
[root@master-a ~]#
```

```
[root@haproxy-client ~]# sudo tcpdump -i any port 3306
tcpdump: data link type LINUX_SLL2
dropped privs to tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
03:25:28.614849 ens33 Out IP haproxy-client.51042 > 192.168.117.140.mysql: Flags [S], seq 250179428, win 64240, options [mss 1460,sackOK,TS val 1559932882 ecr 0,nop,wscale 7], length 0
03:25:28.615353 ens33 In  IP 192.168.117.140.mysql > haproxy-client.51042: Flags [S.], seq 1152003319, ack 250179429, win 65160, options [mss 1460,sackOK,TS val 3689639009 ecr 15599328
82,nop,wscale 7], length 0
03:25:28.615420 ens33 Out IP haproxy-client.51042 > 192.168.117.140.mysql: Flags [R.], seq 1, ack 1, win 502, options [nop,nop,TS val 1559932883 ecr 3689639009], length 0
03:25:29.522423 ens33 Out IP haproxy-client.55646 > 192.168.117.139.mysql: Flags [S], seq 2082515992, win 64240, options [mss 1460,sackOK,TS val 1392976059 ecr 0,nop,wscale 7], length
0
03:25:29.523829 ens33 In  IP 192.168.117.139.mysql > haproxy-client.55646: Flags [S.], seq 3725414063, ack 2082515993, win 65160, options [mss 1460,sackOK,TS val 469872014 ecr 13929760
59,nop,wscale 7], length 0
03:25:29.524035 ens33 Out IP haproxy-client.55646 > 192.168.117.139.mysql: Flags [R.], seq 1, ack 1, win 502, options [nop,nop,TS val 1392976061 ecr 469872014], length 0
03:25:30.618065 ens33 Out IP haproxy-client.51044 > 192.168.117.140.mysql: Flags [S], seq 1877133124, win 64240, options [mss 1460,sackOK,TS val 1559934886 ecr 0,nop,wscale 7], length
0
03:25:30.619865 ens33 In  IP 192.168.117.140.mysql > haproxy-client.51044: Flags [S.], seq 1752194514, ack 1877133125, win 65160, options [mss 1460,sackOK,TS val 3689641013 ecr 1559934
886,nop,wscale 7], length 0
03:25:30.620216 ens33 Out IP haproxy-client.51044 > 192.168.117.140.mysql: Flags [R.], seq 1, ack 1, win 502, options [nop,nop,TS val 1559934888 ecr 3689641013], length 0
03:25:31.526630 ens33 Out IP haproxy-client.55648 > 192.168.117.139.mysql: Flags [S], seq 312441435, win 64240, options [mss 1460,sackOK,TS val 1392978063 ecr 0,nop,wscale 7], length 0
03:25:31.527127 ens33 In  IP 192.168.117.139.mysql > haproxy-client.55648: Flags [S.], seq 208394311, ack 312441436, win 65160, options [mss 1460,sackOK,TS val 469874018 ecr 1392978063
,nop,wscale 7], length 0
03:25:31.527203 ens33 Out IP haproxy-client.55648 > 192.168.117.139.mysql: Flags [R.], seq 1, ack 1, win 502, options [nop,nop,TS val 1392978064 ecr 469874018], length 0
03:25:32.620971 ens33 Out IP haproxy-client.51046 > 192.168.117.140.mysql: Flags [S], seq 1391823527, win 64240, options [mss 1460,sackOK,TS val 1559936889 ecr 0,nop,wscale 7], length
0
03:25:32.622788 ens33 In  IP 192.168.117.140.mysql > haproxy-client.51046: Flags [S.], seq 1273529099, ack 1391823528, win 65160, options [mss 1460,sackOK,TS val 3689643016 ecr 1559936
889,nop,wscale 7], length 0
03:25:32.623190 ens33 Out IP haproxy-client.51046 > 192.168.117.140.mysql: Flags [R.], seq 1, ack 1, win 502, options [nop,nop,TS val 1559936891 ecr 3689643016], length 0
03:25:33.529790 ens33 Out IP haproxy-client.55650 > 192.168.117.139.mysql: Flags [S], seq 1948646565, win 64240, options [mss 1460,sackOK,TS val 1392980066 ecr 0,nop,wscale 7], length
0
03:25:33.531610 ens33 In  IP 192.168.117.139.mysql > haproxy-client.55650: Flags [S.], seq 2018913012, ack 1948646566, win 65160, options [mss 1460,sackOK,TS val 469876022 ecr 13929800
66,nop,wscale 7], length 0
03:25:33.531949 ens33 Out IP haproxy-client.55650 > 192.168.117.139.mysql: Flags [R.], seq 1, ack 1, win 502, options [nop,nop,TS val 1392980069 ecr 469876022], length 0
03:25:34.624198 ens33 Out IP haproxy-client.51048 > 192.168.117.140.mysql: Flags [S], seq 701962302, win 64240, options [mss 1460,sackOK,TS val 1559938892 ecr 0,nop,wscale 7], length 0
03:25:34.625885 ens33 In  IP 192.168.117.140.mysql > haproxy-client.51048: Flags [S.], seq 952186573, ack 701962303, win 65160, options [mss 1460,sackOK,TS val 3689645019 ecr 155993889
2,nop,wscale 7], length 0
03:25:34.626276 ens33 Out IP haproxy-client.51048 > 192.168.117.140.mysql: Flags [R.], seq 1, ack 1, win 502, options [nop,nop,TS val 1559938894 ecr 3689645019], length 0
03:25:35.534239 ens33 Out IP haproxy-client.55652 > 192.168.117.139.mysql: Flags [S], seq 1763381524, win 64240, options [mss 1460,sackOK,TS val 1392982071 ecr 0,nop,wscale 7], length
0
```

**How to read the columns:**

1. Netid: The protocol type (tcp or udp).

2. State: For TCP, LISTEN means the port is open and waiting. For UDP, it will say UNCONN (Unconnected) because UDP doesn't maintain state connections.

3. Local Address:Port: The network interface and port your machine is listening on.

    - 0.0.0.0:22 means the service (SSH) is listening on all available network interfaces on port 22.

- 127.0.0.1:3306 means the service (MySQL) is strictly tied to the localhost loopback adapter, meaning it only accepts connections originating from inside this specific machine.

4. Peer Address: Port: The remote address allowed to connect. 0.0.0.0: means it will accept traffic from any remote port/IP.

5. Process: The specific program name and its system PID. In the example above, you can easily tell that Process ID 1012 is your mysqld database engine running on port 3306.

**Common Use Cases**

System administrators and network engineers use this exact command constantly to:

- Verify if a service (like Apache, Nginx, or MySQL) started up correctly and bound itself to the right port.

- Audit local system security to ensure no unexpected background applications are exposing open entryways to the network.

- Troubleshoot port conflicts (e.g., finding out what process is already sitting on port 80 so you can terminate it).

## Example of Securing my.cnf (SSL Keys)

```
[mysqld]


server-id=1
log_bin=mysql-bin
binlog_format=ROW
gtid_mode=ON
enforce_gtid_consistency=ON
log_slave_updates=ON
read_only=OFF
auto_increment_increment=2
auto_increment_offset=1


validate_password.policy=STRONG
validate_password.length=12


require_secure_transport=ON
ssl-ca=ca.pem
ssl-cert=server-cert.pem
ssl-key=server-key.pem
```

## Secure Login for Clients with SSL keys

```
[root@master-a ~]# mysql -u admin_user -p --ssl-mode=REQUIRED
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 32
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

You are enforcing ssl connection via unix socket. Please consider
switching ssl off as it does not make connection via unix socket
any more secure.
mysql>
```

## Audit User Privileges and Password Policies

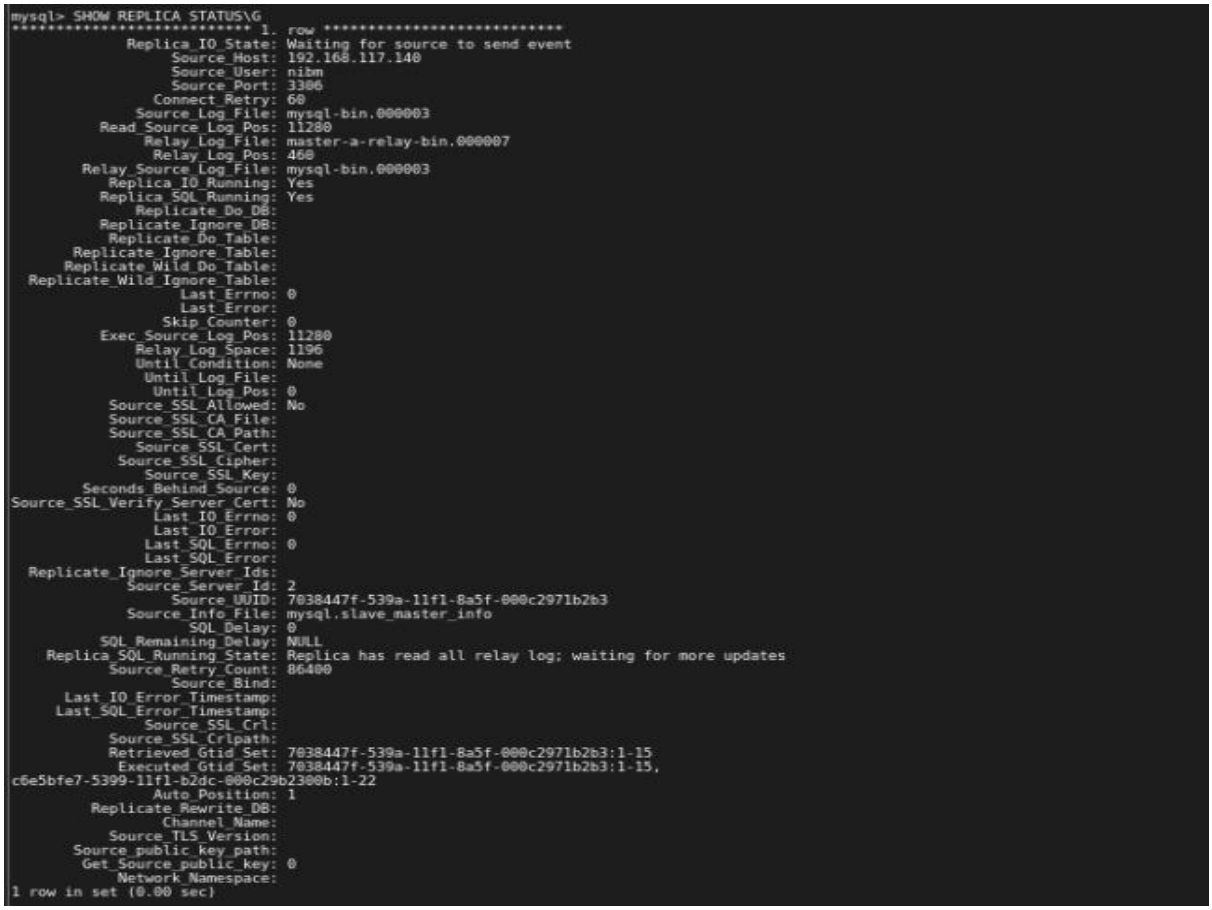
```
mysql> SELECT user, host FROM mysql.user;
No connection. Trying to reconnect...
Connection id:    14
Current database: secure_demo

+------------------+-----------+
| user             | host      |
+------------------+-----------+
| admin_role       | %         |
| admin_user       | %         |
| auditor_role     | %         |
| auditor_user     | %         |
| backup_role      | %         |
| backup_user      | %         |
| haproxy          | %         |
| nibm             | %         |
| pavan            | %         |
| staff_role       | %         |
| staff_user       | %         |
| mysql.infoschema | localhost |
| mysql.session    | localhost |
| mysql.sys        | localhost |
| root             | localhost |
+------------------+-----------+
15 rows in set (0.00 sec)

mysql> SHOW GRANTS FOR 'staff_user'@'%';
+-------------------------------------------------------+
| Grants for staff_user@%                               |
+-------------------------------------------------------+
| GRANT USAGE ON *.* TO `staff_user`@`%`                |
| GRANT SELECT ON `enterprise_secure_db`.* TO `staff_user`@`%` |
| GRANT `staff_role`@`%` TO `staff_user`@`%`            |
+-------------------------------------------------------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM performance_schema.host_cache;
+----------------+------+----------------+-------------------+-------------------------+-------------------------+-------------------------+-------------------+-----------------------+-----------------+-----------------+-----
```

**Audit Replication Security**




**Regulatory Compliance Framework Mapping**

To ensure the system complies with international regulatory standards, our database auditing setup is mapped to specific clauses within major frameworks below:

**ISO/IEC 27001:2022 Control Mapping**

- Control A.8.15 (Logging): Requires systems to generate event logs recording user activities, exceptions, and security events, and mandates that these logs be protected and regularly reviewed.

    - Compliance Implementation: Enforced by setting audit_log_policy=ALL, which creates a permanent, timestamped record of all data queries and configuration adjustments.

**PCI DSS v4.0 (Payment Card Industry Data Security Standard)**

- Requirement 10.2.1: Mandates the implementation of automated audit trails to record all individual access to systems containing sensitive cardholder data environments (CDE).

    - Compliance Implementation: The structured JSON engine captures every transactional query, recording the exact user identity, timestamp, and query details to fulfill payment processing regulations.

**GDPR (General Data Protection Regulation)**

- Article 5(2) (Accountability Principle): Mandates that organizations must be able to demonstrate that personal data is processed securely and protected against unauthorized or accidental processing.

    - Compliance Implementation: The immutable tracking of data events within our local log file creates an active, transparent record of personal data processing activities, ensuring regulatory compliance.

| Compliance Standard | Target Requirement Clause | Technical System Control Implemented | Audit Verification Verdict |
|---|---|---|---|
| ISO 27001:2022 | Control A.8.15 (System Logging) | Installed the server-side `audit_log` library module to capture real-time user activity. | COMPLIANT (Verified via ACTIVE plugin state) |
| PCI DSS v4.0 | Requirement 10.2.1 (Audit Trails) | Configured `audit_log_policy=ALL` to catch and document all executed backend queries. | COMPLIANT (Verified via JSON query logs) |
| GDPR | Article 5(2) (Accountability Principle) | Created an immutable log file path tracking user signatures and timestamps for data events. | COMPLIANT (Verified via captured DROP event) |